CMSE 202 Midterm reference (Section 001, Fall 2026)

Copied from the published documentation. Each page starts with the address it comes from.

Retrieved 2026-10-10.

Git

- git clone
- git status
- git add
- git commit
- git push
- git log

Command line

- mkdir
- mv
- curl
- pip install

pandas

- pandas.read_csv
- DataFrame.head
- DataFrame.shape
- DataFrame.isna
- DataFrame.sum
- DataFrame.dropna
- DataFrame.groupby
- DataFrameGroupBy.size
- DataFrameGroupBy.median
- Boolean indexing

matplotlib

- pyplot.scatter
- pyplot.xscale
- pyplot.xlabel
- pyplot.title
- pyplot.legend

astropy.units

- Getting started
- Quantity
- Enabling other units

Python

- Defining functions
- Dictionaries
- A first look at classes

Git

Contents | Source: https://git-scm.com/docs/git-clone
Git reference manual. Git and its documentation are licensed under the GNU General Public License, version 2.

NAME

git-clone - Clone a repository into a new directory

SYNOPSIS

```
git clone [--template=<template-directory>]
          [-l] [-s] [--no-hardlinks] [-q] [-n] [--bare] [--mirror]
          [-o <name>] [-b <name>] [-u <upload-pack>] [--reference <repository>]
          [--dissociate] [--separate-git-dir <git-dir>]
          [--depth <depth>] [--[no-]single-branch] [--[no-]tags]
          [--recurse-submodules[=<pathspec>]] [--[no-]shallow-submodules]
          [--[no-]remote-submodules] [--jobs <n>] [--sparse] [--[no-]reject-shallow]
          [--filter=<filter-spec> [--also-filter-submodules]] [--] <repository>
          [<directory>]
```

DESCRIPTION

Clones a repository into a newly created directory, creates remote-tracking branches for each branch in the cloned repository (visible using `git branch --remotes`), and creates and checks out an initial branch that is forked from the cloned repository's currently active branch.

Contents | Source: https://git-scm.com/docs/git-status
Git reference manual. Git and its documentation are licensed under the GNU General Public License, version 2.

NAME

git-status - Show the working tree status

SYNOPSIS

```
git status [<options>] [--] [<pathspec>…]
```

DESCRIPTION

Displays paths that have differences between the index file and the current HEAD commit, paths that have differences between the working tree and the index file, and paths in the working tree that are not tracked by Git (and are not ignored by gitignore[5]). The first are what you *would* commit by running `git commit`; the second and third are what you *could* commit by running `git add` before running `git commit`.

Contents | Source: https://git-scm.com/docs/git-add
Git reference manual. Git and its documentation are licensed under the GNU General Public License, version 2.

NAME

git-add - Add file contents to the index

SYNOPSIS

```
git add [--verbose | -v] [--dry-run | -n] [--force | -f] [--interactive | -i] [--patch | -p]
        [--edit | -e] [--[no-]all | -A | --[no-]ignore-removal | [--update | -u]] [--sparse]
        [--intent-to-add | -N] [--refresh] [--ignore-errors] [--ignore-missing] [--renormalize]
        [--resolved] [--chmod=(+|-)x] [--pathspec-from-file=<file> [--pathspec-file-nul]]
        [--] [<pathspec>…]
```

DESCRIPTION

Add contents of new or changed files to the index. The "index" (also known as the "staging area") is what you use to prepare the contents of the next commit.

Contents | Source: https://git-scm.com/docs/git-commit
Git reference manual. Git and its documentation are licensed under the GNU General Public License, version 2.

NAME

git-commit - Record changes to the repository

SYNOPSIS

```
git commit [-a | --interactive | --patch] [-s] [-v] [-u[<mode>]] [--amend]
           [--dry-run] <commit>_ | --fixup [(amend|reword):"><commit>]
           [-F <file> | -m <msg>] [--reset-author] [--allow-empty]
           [--allow-empty-message] [--no-verify] [-e] [--author=<author>]
           [--date=<date>] [--cleanup=<mode>] [--[no-]status]
           [-i | -o] [--pathspec-from-file=<file> [--pathspec-file-nul]]
           [(--trailer <token>[(=|:)<value>])…] [-S[<keyid>]]
           [--] [<pathspec>…]
```

DESCRIPTION

Create a new commit containing the current contents of the index and the given log message describing the changes. The new commit is a direct child of HEAD, usually the tip of the current branch, and the branch is updated to point to it (unless no branch is associated with the working tree, in which case `HEAD` is "detached" as described in git-checkout[1]).

OPTIONS

`-m` *<msg>*

`--message=`*<msg>*

> Use *<msg>* as the commit message. If multiple `-m` options are given, their values are concatenated as separate paragraphs.
>
> The `-m` option is mutually exclusive with `-c`, `-C`, and `-F`.

Contents | Source: https://git-scm.com/docs/git-push
Git reference manual. Git and its documentation are licensed under the GNU General Public License, version 2.

NAME

git-push - Update remote refs along with associated objects

SYNOPSIS

```
git push [--all | --branches | --mirror | --tags] [--follow-tags] [--atomic] [-n | --dry-run] [--
receive-pack=<git-receive-pack>]
         [--repo=<repository>] [-f | --force] [-d | --delete] [--prune] [-q | --quiet] [-v | --
verbose]
         [-u | --set-upstream] [-o <string> | --push-option=<string>]
         [--[no-]signed | --signed=(true|false|if-asked)]
         [--force-with-lease[=<refname>[:<expect>]] [--force-if-includes]]
         [--no-verify] [<repository> [<refspec>…]]
```

DESCRIPTION

Updates one or more branches, tags, or other references in one or more remote repositories from your local repository, and sends all necessary data that isn't already on the remote.

Contents | Source: https://git-scm.com/docs/git-log
Git reference manual. Git and its documentation are licensed under the GNU General Public License, version 2.

NAME

git-log - Show commit logs

SYNOPSIS

```
git log [<options>] [<revision-range>] [[--] <path>…]
```

DESCRIPTION

Shows the commit logs.

OPTIONS

`--oneline`

> This is a shorthand for `--pretty=oneline --abbrev-commit` used together.

Command line

Contents | Source: https://man7.org/linux/man-pages/man1/mkdir.1.html
Copyright © 2026 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later < https://gnu.org/licenses/gpl.html >.
This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law.

NAME

```
mkdir - make directories
```

SYNOPSIS

```
mkdir [OPTION]... DIRECTORY...
```

COPYRIGHT

```
Copyright © 2026 Free Software Foundation, Inc.  License GPLv3+:
GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Contents | Source: https://man7.org/linux/man-pages/man1/mv.1.html
Copyright © 2026 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later < https://gnu.org/licenses/gpl.html >.
This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law.

NAME

```
mv - move (rename) files
```

SYNOPSIS

```
mv [OPTION]... [-T] SOURCE DEST
mv [OPTION]... SOURCE... DIRECTORY
mv [OPTION]... -t DIRECTORY SOURCE...
```

COPYRIGHT

```
Copyright © 2026 Free Software Foundation, Inc.  License GPLv3+:
GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Contents | Source: https://curl.se/docs/manpage.html
curl documentation. Copyright (c) 1996 - 2026, Daniel Stenberg, daniel@haxx.se, and many contributors. curl is licensed under the curl license.

Synopsis

curl [options / URLs]

-o, --output <file>

Write output to the given file instead of stdout. If you are using globbing in the URL to fetch multiple documents, you should quote the URL and you can use "#" followed by a number in the filename. That variable gets replaced with the current glob text. Like in:

-O, --remote-name

Write output to a local file named like the remote file we get. (Only the file part of the remote file is used, the path is cut off.)

Contents | Source: https://pip.pypa.io/en/stable/cli/pip_install/
Copyright © The pip developers

Usage

◉ Unix/macOS

```
python -m pip install [options] <requirement specifier> [package-index-options] ...
python -m pip install [options] -r <requirements file> [package-index-options] ...
python -m pip install [options] [-e] <vcs project url> ...
python -m pip install [options] [-e] <local project path> ...
python -m pip install [options] <archive url/path> ...
```

○ Windows

```
py -m pip install [options] <requirement specifier> [package-index-options] ...
py -m pip install [options] -r <requirements file> [package-index-options] ...
py -m pip install [options] [-e] <vcs project url> ...
py -m pip install [options] [-e] <local project path> ...
py -m pip install [options] <archive url/path> ...
```

Options

--user

Install to the Python user install directory for your platform. Typically ~/.local/, or %APPDATA%Python on Windows. (See the Python documentation for site.USER_BASE for full details.)

(environment variable: `PIP_USER`)

pandas

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.read_csv

pandas.read_csv(*filepath_or_buffer*, *, *sep=<no_default>*, *delimiter=None*, *header='infer'*, *names=<no_default>*, *index_col=None*, *usecols=None*, *dtype=None*, *engine=None*, *converters=None*, *true_values=None*, *false_values=None*, *skipinitialspace=False*, *skiprows=None*, *skipfooter=0*, *nrows=None*, *na_values=None*, *keep_default_na=True*, *na_filter=True*, *skip_blank_lines=True*, *parse_dates=None*, *date_format=None*, *dayfirst=False*, *cache_dates=True*, *iterator=False*, *chunksize=None*, *compression='infer'*, *thousands=None*, *decimal='.'*, *lineterminator=None*, *quotechar='"'*, *quoting=0*, *doublequote=True*, *escapechar=None*, *comment=None*, *encoding=None*, *encoding_errors='strict'*, *dialect=None*, *on_bad_lines='error'*, *low_memory=True*, *memory_map=False*, *float_precision=None*, *storage_options=None*, *dtype_backend=<no_default>*)[source]

Read a comma-separated values (csv) file into DataFrame.

Parameters:

filepath_or_bufferstr, path object or file-like object

Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: file://localhost/path/to/table.csv.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

sepstr, default ‘,’

Character or regex pattern to treat as the delimiter. `sep=None` detects the separator from the first valid row of the file with Python’s builtin sniffer tool, `csv.Sniffer`; it is supported only by the Python parsing engine, which will be used automatically. In addition, separators longer than 1 character and different from `'\s+'` will be interpreted as regular expressions and will also force the use of the Python parsing engine. Note that regex delimiters are prone to ignoring quoted data. Regex example: `'\r\t'`.

Returns:

DataFrame or TextFileReader

A comma-separated values (csv) file is returned as two-dimensional data structure with labeled axes.

Examples

```
>>> pd.read_csv("data.csv")
   Name  Value
0   foo      1
1   bar      2
2  #baz      3
```

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.head.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.DataFrame.head

DataFrame.head(*n=5*)[source]

Return the first *n* rows.

Parameters:

nint, default 5

Number of rows to select.

Returns:

same type as caller

The first *n* rows of the caller object.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "animal": [
...             "alligator",
...             "bee",
...             "falcon",
...             "lion",
...             "monkey",
...             "parrot",
...             "shark",
...             "whale",
...             "zebra",
...         ]
...     }
... )
>>> df
      animal
0  alligator
1        bee
2     falcon
3       lion
4     monkey
5     parrot
6      shark
7      whale
8      zebra
```

Viewing the first 5 lines

```
>>> df.head()
      animal
0  alligator
1        bee
2     falcon
3       lion
4     monkey
```

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.shape.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.DataFrame.shape

property DataFrame.shape[source]

Return a tuple representing the dimensionality of the DataFrame.

Examples

```
>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df.shape
(2, 2)
```

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.isna.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.DataFrame.isna

DataFrame.isna()[source]

Detect missing values.

Returns:

Series/DataFrame

Mask of bool values for each element in Series/DataFrame that indicates whether an element is an NA value.

Examples

Show which entries in a DataFrame are NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age       born    name        toy
0  5.0        NaT  Alfred        NaN
1  6.0 1939-05-27  Batman  Batmobile
2  NaN 1940-04-25              Joker
```

```
>>> df.isna()
     age   born   name    toy
0  False   True  False   True
1  False  False  False  False
2   True  False  False  False
```

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sum.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.DataFrame.sum

DataFrame.sum(*, *axis=0*, *skipna=True*, *numeric_only=False*, *min_count=0*, ***kwargs*)[source]

Return the sum of the values over the requested axis.

Parameters:

axis{index (0), columns (1)}

Axis for the function to be applied on. For *Series* this parameter is unused and defaults to 0.

> Warning
>
> The behavior of DataFrame.sum with `axis=None` is deprecated, in a future version this will reduce over both axes and return a scalar To retain the old behavior, pass axis=0 (or do not pass axis).

Added in version 2.0.0.

Returns:

Series or scalar

Sum over requested axis.

Examples

```
>>> idx = pd.MultiIndex.from_arrays(
...     [["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
...     names=["blooded", "animal"],
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded  animal
warm     dog       4
         falcon    2
cold     fish      0
         spider    8
Name: legs, dtype: int64
```

```
>>> s.sum()
14
```

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.DataFrame.dropna

DataFrame.dropna(*, *axis=0*, *how=<no_default>*, *thresh=<no_default>*, *subset=None*, *inplace=False*, *ignore_index=False*)[source]

Remove missing values.

Parameters:

axis{0 or ‘index’, 1 or ‘columns’}, default 0

Determine if rows or columns which contain missing values are removed.

- 0, or ‘index’ : Drop rows which contain missing values.
- 1, or ‘columns’ : Drop columns which contain missing value.

Only a single axis is allowed.

how{‘any’, ‘all’}, default ‘any’

Determine if row or column is removed from DataFrame, when we have at least one NA or all NA.

- ‘any’ : If any NA values are present, drop that row or column.
- ‘all’ : If all values are NA, drop that row or column.

subsetcolumn label or iterable of labels, optional

Labels along other axis to consider, e.g. if you are dropping rows these would be a list of columns to include.

Returns:

DataFrame or None

DataFrame with NA entries dropped from it or None if `inplace=True`.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "name": ["Alfred", "Batman", "Catwoman"],
...         "toy": [np.nan, "Batmobile", "Bullwhip"],
...         "born": [pd.NaT, pd.Timestamp("1940-04-25"), pd.NaT],
...     }
... )
>>> df
       name        toy       born
0    Alfred        NaN        NaT
1    Batman  Batmobile 1940-04-25
2  Catwoman   Bullwhip        NaT
```

Drop the rows where at least one element is missing.

```
>>> df.dropna()
     name        toy       born
1  Batman  Batmobile 1940-04-25
```

Drop the columns where at least one element is missing.

```
>>> df.dropna(axis="columns")
       name
0    Alfred
1    Batman
2  Catwoman
```

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.DataFrame.groupby

DataFrame.groupby(*by=None*, *level=None*, *, *as_index=True*, *sort=True*, *group_keys=True*, *observed=True*, *dropna=True*)[source]**

Group DataFrame using a mapper or by a Series of columns.

Parameters:

bymapping, function, label, pd.Grouper or list of such

Used to determine the groups for the groupby. If `by` is a function, it's called on each value of the object's index. If a dict or Series is passed, the Series or dict VALUES will be used to determine the groups (the Series' values are first aligned; see `.align()` method). If a list or ndarray of length equal to the number of rows is passed (see the groupby user guide), the values are used as-is to determine the groups. A label or list of labels may be passed to group by the columns in `self`. Notice that a tuple is interpreted as a (single) key.

Returns:

pandas.api.typing.DataFrameGroupBy

Returns a groupby object that contains information about the groups.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "Animal": ["Falcon", "Falcon", "Parrot", "Parrot"],
...         "Max Speed": [380.0, 370.0, 24.0, 26.0],
...     }
... )
>>> df
   Animal  Max Speed
0  Falcon      380.0
1  Falcon      370.0
2  Parrot       24.0
3  Parrot       26.0
>>> df.groupby(["Animal"]).mean()
        Max Speed
Animal
Falcon      375.0
Parrot       25.0
```

Hierarchical Indexes

We can groupby different levels of a hierarchical index using the *level* parameter:

```
>>> arrays = [
...     ["Falcon", "Falcon", "Parrot", "Parrot"],
...     ["Captive", "Wild", "Captive", "Wild"],
... ]
>>> index = pd.MultiIndex.from_arrays(arrays, names=("Animal", "Type"))
>>> df = pd.DataFrame({"Max Speed": [390.0, 350.0, 30.0, 20.0]}, index=index)
>>> df
                Max Speed
Animal Type
Falcon Captive      390.0
       Wild         350.0
Parrot Captive       30.0
       Wild          20.0
>>> df.groupby(level=0).mean()
        Max Speed
Animal
Falcon      370.0
Parrot       25.0
>>> df.groupby(level="Type").mean()
         Max Speed
Type
Captive      210.0
Wild         185.0
```

Contents | Source: https://pandas.pydata.org/docs/reference/api/pandas.api.typing.DataFrameGroupBy.size.html
© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.api.typing.DataFrameGroupBy.size

DataFrameGroupBy.size()[source]

Compute group sizes.

Returns:

DataFrame or Series

Number of rows in each group as a Series if as_index is True or a DataFrame if as_index is False.

Examples

For SeriesGroupBy:

```
>>> lst = ["a", "a", "b"]
>>> ser = pd.Series([1, 2, 3], index=lst)
>>> ser
a     1
a     2
b     3
dtype: int64
>>> ser.groupby(level=0).size()
a    2
b    1
dtype: int64
```


© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

pandas.api.typing.DataFrameGroupBy.median

DataFrameGroupBy.median(*numeric_only=False*, *skipna=True*)[source]

Compute median of groups, excluding missing values.

Returns:

Series or DataFrame

Median of values within each group.

Examples

For SeriesGroupBy:

```
>>> lst = ["a", "a", "a", "b", "b", "b"]
>>> ser = pd.Series([7, 2, 8, 4, 3, 3], index=lst)
>>> ser
a     7
a     2
a     8
b     4
b     3
b     3
dtype: int64
>>> ser.groupby(level=0).median()
a    7.0
b    3.0
dtype: float64
```


© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud .

Boolean indexing

Another common operation is the use of boolean vectors to filter the data. The operators are: `|` for `or`, `&` for `and`, and `~` for `not`. These **must** be grouped by using parentheses, since by default Python will evaluate an expression such as `df['A'] > 2 & df['B'] < 3` as `df['A'] > (2 & df['B']) < 3`, while the desired evaluation order is `(df['A'] > 2) & (df['B'] < 3)`.

Using a boolean vector to index a Series works exactly as in a NumPy ndarray:

```
In [168]: s = pd.Series(range(-3, 4))

In [169]: s
Out[169]:
0   -3
1   -2
2   -1
3    0
4    1
5    2
6    3
dtype: int64

In [170]: s[s > 0]
Out[170]:
4    1
5    2
6    3
dtype: int64

In [171]: s[(s < -1) | (s > 0.5)]
Out[171]:
0   -3
1   -2
4    1
5    2
6    3
dtype: int64

In [172]: s[~(s < 0)]
Out[172]:
3    0
4    1
5    2
6    3
dtype: int64
```

You may select rows from a DataFrame using a boolean vector the same length as the DataFrame's index (for example, something derived from one of the columns of the DataFrame):

```
In [173]: df[df['A'] > 0]
Out[173]:
                   A         B         C         D    E   0
2000-01-04  7.000000  0.721555 -1.039575  0.271860  NaN NaN
2000-01-05  0.567020 -0.424972  0.276232 -1.087401  NaN NaN
2000-01-06  0.113648 -0.673690 -1.478427  0.524988  7.0 NaN
2000-01-07  0.577046  0.404705 -1.715002 -1.039268  NaN NaN
```

matplotlib

Contents | Source: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.scatter.html
© Copyright 2002–2012 John Hunter, Darren Dale, Eric Firing, Michael Droettboom and the Matplotlib development team; 2012–2026 The Matplotlib development team.

matplotlib.pyplot.scatter

matplotlib.pyplot.scatter(*x*, *y*, *s=None*, *c=None*, *, *marker=None*, *cmap=None*, *norm=None*, *vmin=None*, *vmax=None*, *alpha=None*, *linewidths=None*, *edgecolors=None*, *colorizer=None*, *plotnonfinite=False*, *data=None*, ***kwargs*)[source]

A scatter plot of *y* vs. *x* with varying marker size and/or color.

Parameters:

x, yfloat or array-like, shape (n,)

The data positions.

carray-like or list of color or color, optional

The marker colors. Possible values:

- A scalar or sequence of n numbers to be mapped to colors using *cmap* and *norm*.
- A 2D array in which the rows are RGB or RGBA.
- A sequence of colors of length n.
- A single color format string.

Note that *c* should not be a single numeric RGB or RGBA sequence because that is indistinguishable from an array of values to be colormapped. If you want to specify the same RGB or RGBA value for all points, use a 2D array with a single row. Otherwise, value-matching will have precedence in case of a size matching with *x* and *y*.

If you wish to specify a single color for all points prefer the *color* keyword argument.

Defaults to `None`. In that case the marker color is determined by the value of *color*, *facecolor* or *facecolors*. In case those are not specified or `None`, the marker color is determined by the next color of the `Axes`' current "shape and fill" color cycle. This cycle defaults to `rcParams["axes.prop_cycle"]` (default: `cycler('color', [(0.12156862745098039, 0.4666666666666667, 0.7058823529411765), (1.0, 0.4980392156862745, 0.054901960784313725), (0.17254901960784313, 0.6274509803921569, 0.17254901960784313), (0.8392156862745098, 0.15294117647058825, 0.1568627450980392), (0.5803921568627451, 0.403921568627451, 0.7411764705882353), (0.5490196078431373, 0.33725490196078434, 0.29411764705882354), (0.8901960784313725, 0.4666666666666667, 0.7607843137254902), (0.4980392156862745, 0.4980392156862745, 0.4980392156862745), (0.7372549019607844, 0.7411764705882353, 0.13333333333333333), (0.09019607843137255, 0.7450980392156863, 0.8117647058823529)])`).

alphafloat, default: None

The alpha blending value, between 0 (transparent) and 1 (opaque).

Returns:

`PathCollection`

Contents | Source: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xscale.html
© Copyright 2002–2012 John Hunter, Darren Dale, Eric Firing, Michael Droettboom and the Matplotlib development team; 2012–2026 The Matplotlib development team.

matplotlib.pyplot.xscale

matplotlib.pyplot.xscale(*value*, ***kwargs*)[source]

Set the xaxis' scale.

Parameters:

valuestr or `ScaleBase`

The axis scale type to apply. Valid string values are the names of scale classes ("linear", "log", "function",...). These may be the names of any of the built-in scales or of any custom scales registered using `matplotlib.scale.register_scale`.

****kwargs**

If *value* is a string, keywords are passed to the instantiation method of the respective class.

Contents | Source: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xlabel.html
© Copyright 2002–2012 John Hunter, Darren Dale, Eric Firing, Michael Droettboom and the Matplotlib development team; 2012–2026 The Matplotlib development team.

matplotlib.pyplot.xlabel

matplotlib.pyplot.xlabel(*xlabel*, *fontdict=None*, *labelpad=None*, *, *loc=None*, ***kwargs*)[source]

Set the label for the x-axis.

Parameters:

xlabelstr

The label text.

Contents | Source: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.title.html
© Copyright 2002–2012 John Hunter, Darren Dale, Eric Firing, Michael Droettboom and the Matplotlib development team; 2012–2026 The Matplotlib development team.

matplotlib.pyplot.title

matplotlib.pyplot.title(*label*, *fontdict=None*, *loc=None*, *pad=None*, *, *y=None*, ***kwargs*)[source]

Set a title for the Axes.

Parameters:

labelstr

Text to use for the title

Returns:

`Text`

The matplotlib text instance representing the title

Contents | Source: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.legend.html
© Copyright 2002–2012 John Hunter, Darren Dale, Eric Firing, Michael Droettboom and the Matplotlib development team; 2012–2026 The Matplotlib development team.

matplotlib.pyplot.legend

matplotlib.pyplot.legend(args*, ***kwargs*)[source]**

Place a legend on the Axes.

Call signatures:

```
legend()
legend(handles, labels)
legend(handles=handles)
legend(labels)
```

The call signatures correspond to the following different ways to use this method:

1. Automatic detection of elements to be shown in the legend

The elements to be added to the legend are automatically determined, when you do not pass in any extra arguments.

In this case, the labels are taken from the artist. You can specify them either at artist creation or by calling the `set_label()` method on the artist:

```
ax.plot([1, 2, 3], label='Inline label')
ax.legend()
```

or:

```
line, = ax.plot([1, 2, 3])
line.set_label('Label via method')
ax.legend()
```

Returns:

`Legend`

astropy.units

Contents | Source: https://docs.astropy.org/en/stable/units/index.html
© Copyright 2011–2026, The Astropy Developers.

Getting Started

Most users of the `astropy.units` package will work with Quantity objects: the combination of a value and a unit. The most convenient way to create a `Quantity` is to multiply or divide a value by one of the built-in units. It works with scalars, sequences, and `numpy` arrays.

Examples

To create a `Quantity` object:

```
>>> from astropy import units as u
>>> 42.0 * u.meter
<Quantity  42. m>
>>> [1., 2., 3.] * u.m
<Quantity [1., 2., 3.] m>
>>> import numpy as np
>>> np.array([1., 2., 3.]) * u.m
<Quantity [1., 2., 3.] m>
```

You can get the unit and value from a `Quantity` using the unit and value members:

```
>>> q = 42.0 * u.meter
>>> q.value
np.float64(42.0)
>>> q.unit
Unit("m")
```

From this basic building block, it is possible to start combining quantities with different units:

```
>>> 15.1 * u.meter / (32.0 * u.second)
<Quantity 0.471875 m / s>
>>> 3.0 * u.kilometer / (130.51 * u.meter / u.second)
<Quantity 0.022986744310780783 km s / m>
>>> (3.0 * u.kilometer / (130.51 * u.meter / u.second)).decompose()
<Quantity 22.986744310780782 s>
```

Unit conversion is done using the `to()` method, which returns a new `Quantity` in the given unit:

```
>>> x = 1.0 * u.parsec
>>> x.to(u.km)
<Quantity 30856775814671.914 km>
```

It is also possible to work directly with units at a lower level, for example, to create custom units:

```
>>> from astropy.units import imperial

>>> cms = u.cm / u.s
>>> # ...and then use some imperial units
>>> mph = imperial.mile / u.hour

>>> # And do some conversions
>>> q = 42.0 * cms
>>> q.to(mph)
<Quantity 0.939513242662849 mi / h>
```

Units that “cancel out” become a special unit called the “dimensionless unit”:

```
>>> u.m / u.m
Unit(dimensionless)
```

To create a basic dimensionless quantity, multiply a value by the unscaled dimensionless unit:

```
>>> q = 1.0 * u.dimensionless_unscaled
>>> q.unit
Unit(dimensionless)
```

`astropy.units` is able to match compound units against the units it already knows about:

```
>>> (u.s ** -1).compose()
[Unit("Bq"), Unit("Hz"), Unit("2.7027e-11 Ci")]
```

And it can convert between unit systems, such as SI or CGS:

```
>>> (1.0 * u.Pa).cgs
<Quantity 10. Ba>
```

The units `mag`, `dex`, and `dB` are special, being logarithmic units, for which a value is the logarithm of a physical quantity in a given unit. These can be used with a physical unit in parentheses to create a corresponding logarithmic quantity:

```
>>> -2.5 * u.mag(u.ct / u.s)
<Magnitude -2.5 mag(ct / s)>
>>> from astropy import constants as c
>>> u.Dex((c.G * u.M_sun / u.R_sun**2).cgs)
<Dex 4.438067627303133 dex(cm / s2)>
```

`astropy.units` also handles equivalencies, such as that between wavelength and frequency. To use that feature, equivalence objects are passed to the `to()` conversion method. For instance, a conversion from wavelength to frequency does not normally work:

```
>>> (1000 * u.nm).to(u.Hz)
Traceback (most recent call last):
  ...
UnitConversionError: 'nm' (length) and 'Hz' (frequency) are not convertible
```

But by passing an equivalency list, in this case `spectral()`, it does:

```
>>> (1000 * u.nm).to(u.Hz, equivalencies=u.spectral())
<Quantity  2.99792458e+14 Hz>
```

Quantities and units can be printed nicely to strings using the Format String Syntax. Format specifiers (like `0.03f`) in strings will be used to format the quantity value:

```
>>> q = 15.1 * u.meter / (32.0 * u.second)
>>> q
<Quantity 0.471875 m / s>
>>> f"{q:0.03f}"
'0.472 m / s'
```

The value and unit can also be formatted separately. Format specifiers for units can be used to choose the unit formatter:

```
>>> q = 15.1 * u.meter / (32.0 * u.second)
>>> q
<Quantity 0.471875 m / s>
>>> f"{q.value:0.03f} {q.unit:FITS}"
'0.472 m s-1'
```

Contents | Source: https://docs.astropy.org/en/stable/units/quantity.html
© Copyright 2011–2026, The Astropy Developers.

Creating Quantity Instances

`Quantity` objects are normally created through multiplication with `Unit` objects.

Examples

To create a `Quantity` to represent 15 m/s:

```
>>> import astropy.units as u
>>> 15 * u.m / u.s
<Quantity 15. m / s>
```

This extends as expected to division by a unit, or using `numpy` arrays or Python sequences:

```
>>> 1.25 / u.s
<Quantity 1.25 1 / s>
>>> [1, 2, 3] * u.m
<Quantity [1., 2., 3.] m>
>>> import numpy as np
>>> np.array([1, 2, 3]) * u.m
<Quantity [1., 2., 3.] m>
```

You can also create instances using the `Quantity` constructor directly, by specifying a value and unit:

```
>>> u.Quantity(15, u.m / u.s)
<Quantity 15. m / s>
```

The constructor gives a few more options. In particular, it allows you to merge sequences of `Quantity` objects (as long as all of their units are equivalent), and to parse simple strings (which may help, for example, to parse configuration files, etc.):

```
>>> qlst = [60 * u.s, 1 * u.min]
>>> u.Quantity(qlst, u.minute)
<Quantity [1.,  1.] min>
>>> u.Quantity('15 m/s')
<Quantity 15. m / s>
```

The current unit and value can be accessed via the `unit` and `value` attributes:

```
>>> q = 2.5 * u.m / u.s
>>> q.unit
Unit("m / s")
>>> q.value
np.float64(2.5)
```

> Note
>
> `Quantity` objects are converted to float by default. Furthermore, any data passed in are copied, which for large arrays may not be optimal. As discussed further below, you can instead obtain a view by passing `copy=False` to `Quantity` or by using the << operator.

Converting to Different Units

`Quantity` objects can be converted to different units using the `to()` method.

Examples

To convert `Quantity` objects to different units:

```
>>> q = 2.3 * u.m / u.s
>>> q.to(u.km / u.h)
<Quantity 8.28 km / h>
```

For convenience, the `si` and `cgs` attributes can be used to convert the `Quantity` to base SI or CGS units:

```
>>> q = 2.4 * u.m / u.s
>>> q.si
<Quantity 2.4 m / s>
>>> q.cgs
<Quantity 240. cm / s>
```

If you want the value of the quantity in a different unit, you can use `to_value()` as a shortcut:

```
>>> q = 2.5 * u.m
>>> q.to_value(u.cm)
np.float64(250.0)
```

> Note
>
> You could get the value in `cm` also by using `q.to(u.cm).value`. The difference is that `to_value()` does no copying if the unit is already the correct one, instead returning a view of the data (just as if you had done `q.value`). In contrast, `to()` always returns a copy (which also means it is slower for the case where no conversion is necessary). As discussed further below, you can avoid the copying by using the << operator.

Addition and Subtraction

Addition or subtraction between `Quantity` objects is supported when their units are equivalent.

Examples

When the units are equal, the resulting object has the same unit:

```
>>> 11 * u.s + 30 * u.s
<Quantity 41. s>
>>> 30 * u.s - 11 * u.s
<Quantity 19. s>
```

If the units are equivalent, but not equal (e.g., kilometer and meter), the resulting object **has units of the object on the left**:

```
>>> 1100.1 * u.m + 13.5 * u.km
<Quantity 14600.1 m>
>>> 13.5 * u.km + 1100.1 * u.m
<Quantity 14.6001 km>
>>> 1100.1 * u.m - 13.5 * u.km
<Quantity -12399.9 m>
>>> 13.5 * u.km - 1100.1 * u.m
<Quantity 12.3999 km>
```

Addition and subtraction are not supported between `Quantity` objects and basic numeric types, except for dimensionless quantities (see Dimensionless Quantities) or special values like zero and infinity:

```
>>> 13.5 * u.km + 19.412
Traceback (most recent call last):
  ...
UnitConversionError: Can only apply 'add' function to dimensionless
quantities when other argument is not a quantity (unless the
latter is all zero/infinity/nan)
```

Contents | Source: https://docs.astropy.org/en/stable/units/standard_units.html
© Copyright 2011–2026, The Astropy Developers.

Enabling Other Units

By default, only the “default” units are searched by `find_equivalent_units()` and similar methods that do searching. This includes SI, CGS, and astrophysical units. However, you may wish to enable the Imperial or other user-defined units.

Example

To enable Imperial units, do:

```
>>> from astropy.units import imperial
>>> imperial.enable()
<astropy.units.core._UnitContext object at ...>
>>> u.m.find_equivalent_units()
  Primary name | Unit definition | Aliases
[
  AU           | 1.49598e+11 m   | au, astronomical_unit            ,
  Angstrom     | 1e-10 m         | AA, angstrom, Å                  ,
  cm           | 0.01 m          | centimeter                       ,
  earthRad     | 6.3781e+06 m    | R_earth, Rearth                  ,
  ft           | 0.3048 m        | foot                             ,
  fur          | 201.168 m       | furlong                          ,
  inch         | 0.0254 m        |                                  ,
  jupiterRad   | 7.1492e+07 m    | R_jup, Rjup, R_jupiter, Rjupiter ,
  lsec         | 2.99792e+08 m   | lightsecond                      ,
  lyr          | 9.46073e+15 m   | lightyear                        ,
  m            | irreducible     | meter                            ,
  mi           | 1609.34 m       | mile                             ,
  micron       | 1e-06 m         |                                  ,
  mil          | 2.54e-05 m      | thou                             ,
  nmi          | 1852 m          | nauticalmile, NM                 ,
  pc           | 3.08568e+16 m   | parsec                           ,
  solRad       | 6.957e+08 m     | R_sun, Rsun                      ,
  yd           | 0.9144 m        | yard                             ,
]
```

This may also be used with the Python “with” statement, to temporarily enable additional units:

```
>>> with imperial.enable():
...     print(u.m.find_equivalent_units())
      Primary name | Unit definition | Aliases
...
```

To enable only specific units, use `add_enabled_units()`:

```
>>> with u.add_enabled_units([imperial.knot]):
...     print(u.m.find_equivalent_units())
      Primary name | Unit definition | Aliases
...
```

Python

Contents | Source: https://docs.python.org/3/tutorial/controlflow.html
© Copyright 2001 Python Software Foundation. This page is licensed under the Python Software Foundation License Version 2. Examples, recipes, and other code in the documentation are additionally licensed under the Zero Clause BSD License. See History and License for more information. The Python Soft

4.8. Defining Functions

We can create a function that writes the Fibonacci series to an arbitrary boundary:

```
>>> def fib(n):    # write Fibonacci series less than n
...     """Print a Fibonacci series less than n."""
...     a, b = 0, 1
...     while a < n:
...         print(a, end=' ')
...         a, b = b, a+b
...     print()
...
>>> # Now call the function we just defined:
>>> fib(2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597
```

The keyword `def` introduces a function *definition*. It must be followed by the function name and the parenthesized list of formal parameters. The statements that form the body of the function start at the next line, and must be indented.

The first statement of the function body can optionally be a string literal; this string literal is the function's documentation string, or *docstring*. (More about docstrings can be found in the section Documentation Strings.) There are tools which use docstrings to automatically produce online or printed documentation, or to let the user interactively browse through code; it's good practice to include docstrings in code that you write, so make a habit of it.

The *execution* of a function introduces a new symbol table used for the local variables of the function. More precisely, all variable assignments in a function store the value in the local symbol table; whereas variable references first look in the local symbol table, then in the local symbol tables of enclosing functions, then in the global symbol table, and finally in the table of built-in names. Thus, global variables and variables of enclosing functions cannot be directly assigned a value within a function (unless, for global variables, named in a `global` statement, or, for variables of enclosing functions, named in a `nonlocal` statement), although they may be referenced.

The actual parameters (arguments) to a function call are introduced in the local symbol table of the called function when it is called; thus, arguments are passed using *call by value* (where the *value* is always an object *reference*, not the value of the object). [1] When a function calls another function, or calls itself recursively, a new local symbol table is created for that call.

A function definition associates the function name with the function object in the current symbol table. The interpreter recognizes the object pointed to by that name as a user-defined function. Other names can also point to that same function object and can also be used to access the function:

```
>>> fib
<function fib at 10042ed0>
>>> f = fib
>>> f(100)
0 1 1 2 3 5 8 13 21 34 55 89
```

Coming from other languages, you might object that `fib` is not a function but a procedure since it doesn't return a value. In fact, even functions without a `return` statement do return a value, albeit a rather boring one. This value is called `None` (it's a built-in name). Writing the value `None` is normally suppressed by the interpreter if it would be the only value written. You can see it if you really want to using `print()`:

```
>>> fib(0)
>>> print(fib(0))
None
```

It is simple to write a function that returns a list of the numbers of the Fibonacci series, instead of printing it:

```
>>> def fib2(n):  # return Fibonacci series up to n
...     """Return a list containing the Fibonacci series up to n."""
...     result = []
...     a, b = 0, 1
...     while a < n:
...         result.append(a)    # see below
...         a, b = b, a+b
...     return result
...
>>> f100 = fib2(100)    # call it
>>> f100                # write the result
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

This example, as usual, demonstrates some new Python features:

- The `return` statement returns with a value from a function. `return` without an expression argument returns `None`. Falling off the end of a function also returns `None`.
- The statement `result.append(a)` calls a *method* of the list object `result`. A method is a function that 'belongs' to an object and is named `obj.methodname`, where `obj` is some object (this may be an expression), and `methodname` is the name of a method that is defined by the object's type. Different types define different methods. Methods of different types may have the same name without causing ambiguity. (It is possible to define your own object types and methods, using *classes*, see Classes) The method `append()` shown in the example is defined for list objects; it adds a new element at the end of the list. In this example it is equivalent to `result = result + [a]`, but more efficient.

Contents | Source: https://docs.python.org/3/tutorial/datastructures.html
© Copyright 2001 Python Software Foundation. This page is licensed under the Python Software Foundation License Version 2. Examples, recipes, and other code in the documentation are additionally licensed under the Zero Clause BSD License. See History and License for more information. The Python Soft

5.5. Dictionaries

Another useful data type built into Python is the *dictionary* (see Mapping types — dict, frozendict). Dictionaries are sometimes found in other languages as "associative memories" or "associative arrays". Unlike sequences, which are indexed by a range of numbers, dictionaries are indexed by *keys*, which can be any immutable type; strings and numbers can always be keys. Tuples can be used as keys if they contain only strings, numbers, or tuples; if a tuple contains any mutable object either directly or indirectly, it cannot be used as a key. You can't use lists as

keys, since lists can be modified in place using index assignments, slice assignments, or methods like `append()` and `extend()`.

It is best to think of a dictionary as a set of *key: value* pairs, with the requirement that the keys are unique (within one dictionary). A pair of braces creates an empty dictionary: `{}`. Placing a comma-separated list of key:value pairs within the braces adds initial key:value pairs to the dictionary; this is also the way dictionaries are written on output.

The main operations on a dictionary are storing a value with some key and extracting the value given the key. It is also possible to delete a key:value pair with `del`. If you store using a key that is already in use, the old value associated with that key is forgotten.

Extracting a value for a non-existent key by subscripting (`d[key]`) raises a `KeyError`. To avoid getting this error when trying to access a possibly non-existent key, use the `get()` method instead, which returns `None` (or a specified default value) if the key is not in the dictionary.

Performing `list(d)` on a dictionary returns a list of all the keys used in the dictionary, in insertion order (if you want it sorted, just use `sorted(d)` instead). To check whether a single key is in the dictionary, use the `in` keyword.

Here is a small example using a dictionary:

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']
4098
>>> tel['irv']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'irv'
>>> print(tel.get('irv'))
None
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False
```

The `dict()` constructor builds dictionaries directly from sequences of key-value pairs:

```
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'guido': 4127, 'jack': 4098}
```

In addition, dict comprehensions can be used to create dictionaries from arbitrary key and value expressions:

```
>>> {x: x**2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
```

And dictionary unpacking (via $**$) can be used to merge multiple dictionaries:

```
>>> odds = {i: i**2 for i in (1, 3, 5)}
>>> evens = {i: i**2 for i in (2, 4, 6)}
>>> {**odds, **evens}
{1: 1, 3: 9, 5: 25, 2: 4, 4: 16, 6: 36}

>>> all_values = [odds, evens, {0: 0}]
>>> {**i for i in all_values}
{1: 1, 3: 9, 5: 25, 2: 4, 4: 16, 6: 36, 0: 0}
```

When the keys are simple strings, it is sometimes easier to specify pairs using keyword arguments:

```
>>> dict(sape=4139, guido=4127, jack=4098)
{'sape': 4139, 'guido': 4127, 'jack': 4098}
```

Contents | Source: https://docs.python.org/3/tutorial/classes.html
© Copyright 2001 Python Software Foundation. This page is licensed under the Python Software Foundation License Version 2. Examples, recipes, and other code in the documentation are additionally licensed under the Zero Clause BSD License. See History and License for more information. The Python Soft

9.3.1. Class Definition Syntax

The simplest form of class definition looks like this:

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

Class definitions, like function definitions (`def` statements) must be executed before they have any effect. (You could conceivably place a class definition in a branch of an `if` statement, or inside a function.)

In practice, the statements inside a class definition will usually be function definitions, but other statements are allowed, and sometimes useful — we'll come back to this later. The function definitions inside a class normally have a peculiar form of argument list, dictated by the calling conventions for methods — again, this is explained later.

When a class definition is entered, a new namespace is created, and used as the local scope — thus, all assignments to local variables go into this new namespace. In particular, function definitions bind the name of the new function here.

When a class definition is left normally (via the end), a *class object* is created. This is basically a wrapper around the contents of the namespace created by the class definition; we'll learn more about class objects in the next section. The original local scope (the one in effect just before the class definition was entered) is reinstated, and the class object is bound here to the class name given in the class definition header (`ClassName` in the example).

9.3.2. Class Objects

Class objects support two kinds of operations: attribute references and instantiation.

Attribute references use the standard syntax used for all attribute references in Python: `obj.name`. Valid attribute names are all the names that were in the class's namespace when the class object was created. So, if the class definition looked like this:

```
class MyClass:
    """A simple example class"""
    i = 12345

    def f(self):
        return 'hello world'
```

then `MyClass.i` and `MyClass.f` are valid attribute references, returning an integer and a function object, respectively. Class attributes can also be assigned to, so you can change the value of `MyClass.i` by assignment. `__doc__` is also a valid attribute, returning the docstring belonging to the class: `"A simple example class"`.

Class *instantiation* uses function notation. Just pretend that the class object is a parameterless function that returns a new instance of the class. For example (assuming the above class):

```
x = MyClass()
```

creates a new *instance* of the class and assigns this object to the local variable `x`.

The instantiation operation (“calling” a class object) creates an empty object. Many classes like to create objects with instances customized to a specific initial state. Therefore a class may define a special method named `__init__()`, like this:

```
def __init__(self):
    self.data = []
```

When a class defines an `__init__()` method, class instantiation automatically invokes `__init__()` for the newly created class instance. So in this example, a new, initialized instance can be obtained by:

```
x = MyClass()
```

Of course, the `__init__()` method may have arguments for greater flexibility. In that case, arguments given to the class instantiation operator are passed on to `__init__()`. For example,

```
>>> class Complex:
...     def __init__(self, realpart, imagpart):
...         self.r = realpart
...         self.i = imagpart
...
>>> x = Complex(3.0, -4.5)
>>> x.r, x.i
(3.0, -4.5)
```

9.3.3. Instance Objects

Now what can we do with instance objects? The only operations understood by instance objects are attribute references. There are two kinds of valid attribute names: data attributes and methods.

Data attributes correspond to “instance variables” in Smalltalk, and to “data members” in C++. Data attributes need not be declared; like local variables, they spring into existence when they are first assigned to. For example, if `x` is the instance of `MyClass` created above, the following piece of code will print the value `16`, without leaving a trace:

```
x.counter = 1
while x.counter < 10:
    x.counter = x.counter * 2
print(x.counter)
del x.counter
```

The other kind of instance attribute reference is a *method*. A method is a function that “belongs to” an object.

Valid method names of an instance object depend on its class. By definition, all attributes of a class that are function objects define corresponding methods of its instances. So in our example, `x.f` is a valid method reference, since `MyClass.f` is a function, but `x.i` is not, since `MyClass.i` is not. But `x.f` is not the same thing as `MyClass.f` — it is a *method object*, not a function object.

9.3.4. Method Objects

Usually, a method is called right after it is bound:

```
x.f()
```

If `x = MyClass()`, as above, this will return the string `'hello world'`. However, it is not necessary to call a method right away: `x.f` is a method object, and can be stored away and called at a later time. For example:

```
xf = x.f
while True:
    print(xf())
```

will continue to print `hello world` until the end of time.

What exactly happens when a method is called? You may have noticed that `x.f()` was called without an argument above, even though the function definition for `f()` specified an argument. What happened to the argument? Surely Python raises an exception when a function that requires an argument is called without any — even if the argument isn’t actually used…

Actually, you may have guessed the answer: the special thing about methods is that the instance object is passed as the first argument of the function. In our example, the call `x.f()` is exactly equivalent to `MyClass.f(x)`. In general, calling a method with a list of *n* arguments is equivalent to calling the corresponding function with an argument list that is created by inserting the method’s instance object before the first argument.

In general, methods work as follows. When a non-data attribute of an instance is referenced, the instance’s class is searched. If the name denotes a valid class attribute that is a function object, references to both the instance object and the function object are packed into a method object. When the method object is called with an argument list, a new argument list is constructed from the instance object and the argument list, and the function object is called with this new argument list.